Adaptive LLM Security & Resource Risk Gateway

Final project report: GPU admission control and short-email injection evaluation

Project operator: Reet — AI-assisted analysis and report preparation

24 October 2026 | Evidence edition 1.0

Contents

1 Executive summary . . . 2
2 Problem, objectives and evaluation questions . . . 2
3 System design and implementation . . . 3
3.1 Native ingress feature vector . . . 3
3.2 Tier and pressure policy . . . 4
4 Experimental environment and evidence provenance . . . 4
5 GPU resource-protection experiment . . . 5
5.1 Workload and controls . . . 5
5.2 Results . . . 5
5.3 Counter reconciliation and cost interpretation . . . 5
5.4 Adaptive-mechanism and latency boundaries . . . 6
6 Security and contextual-model evaluation . . . 7
6.1 Native and specialized diagnostics . . . 7
6.2 Task/content prototype and transfer . . . 7
7 External LLMail short-email study . . . 7
7.1 Source, labels and preparation . . . 7
7.2 Long-input deferral and lexical selection . . . 7
7.3 Fixed training, cutoff selection and reserved results . . . 8
8 Benign checks, labeling policy and reviewer provenance . . . 9
9 Findings, limitations and deferred work . . . 10
10 Conclusion . . . 10
11 Reproducibility and evidence index . . . 11
12 Selected technical references . . . 11

1 Executive summary

This project implements a gateway in front of a GPU-backed large language model (LLM). It extracts lexical, semantic and resource-demand signals, estimates request risk, and assigns full, sandbox or quarantine execution. A Flask gateway forwards admitted requests to the OpenAI-compatible vLLM endpoint serving Qwen2.5-3B-Instruct on an NVIDIA RTX A4000. MiniLM semantic embeddings run on CUDA in FP16, and background telemetry reads actual vLLM KV-cache occupancy.

The strongest measured result is **controlled admission protection**, not general prompt-injection prevention. Across three repetitions of each of direct, static-gateway and adaptive-gateway conditions, mean benign HTTP latency averaged 50.24 seconds direct, 302 milliseconds static and 298 milliseconds adaptive. All 24 benign probes per condition completed. Direct sampled cache occupancy reached 99.56–100%; both protected conditions peaked at 0.881% and blocked all tested high-demand requests. These results use a deliberately constrained **256 MiB KV-cache pool**, eight capped generation demands and eight benign probes per run. They do not establish default-cache out-of-memory prevention, P99 latency, an incremental adaptive advantage, or monetary savings.

The injection investigation produced important negative findings. A task/content pair model scored 59/60 on a synthetic topic test but reviewed only 4/16 redirections under new wording. A subsequent public LLMail short-email experiment used exact-team-component splits, conservative lexical cleanup and frozen operating cutoffs. On 13,989 reserved source-labeled rows, the adjusted pair model achieved 61.59% source-positive recall, 76.66% source-negative specificity and 69.13% balanced accuracy. That operating point did not meet its empirical validation specificity target. The candidate remains offline.

The project therefore delivers a working resource-risk prototype, reproducible evidence and an explicit account of injection-model limitations. **Long-email support is deferred from this submission.** Short-email results are not extrapolated to long documents or complete conversations. A bounded Phase 2 transfer check is being prepared separately; no unobserved result from that check is included in this evidence edition.

2 Problem, objectives and evaluation questions

LLM serving must allocate compute and KV-cache capacity to admitted requests. Excessive generation demand can increase queueing and delay unrelated users. Separately, instructions embedded in externally supplied content may attempt to redirect an assistant away from its authorized task. These problems share an admission boundary but are not interchangeable: a resource-demand detector can protect capacity without reliably recognizing contextual prompt injection.

The project asks four questions:

1. Can the native gateway preserve benign service under a repeatable, capped GPU load?
2. Does the experiment isolate additional benefit from pressure-adaptive thresholds beyond static risk rejection?
3. Do semantic task/content models generalize beyond their development templates and source labels?
4. Can labeling and overlap controls support defensible claims about external data?

Success is assessed from preserved measurements and controls. Architectural targets such as zero GPU cost, zero VRAM delta, sub-2 ms embeddings and universal injection neutralization are not treated as observations. Earlier simulation costs and training-set accuracy are separated from native GPU and reserved-data results.

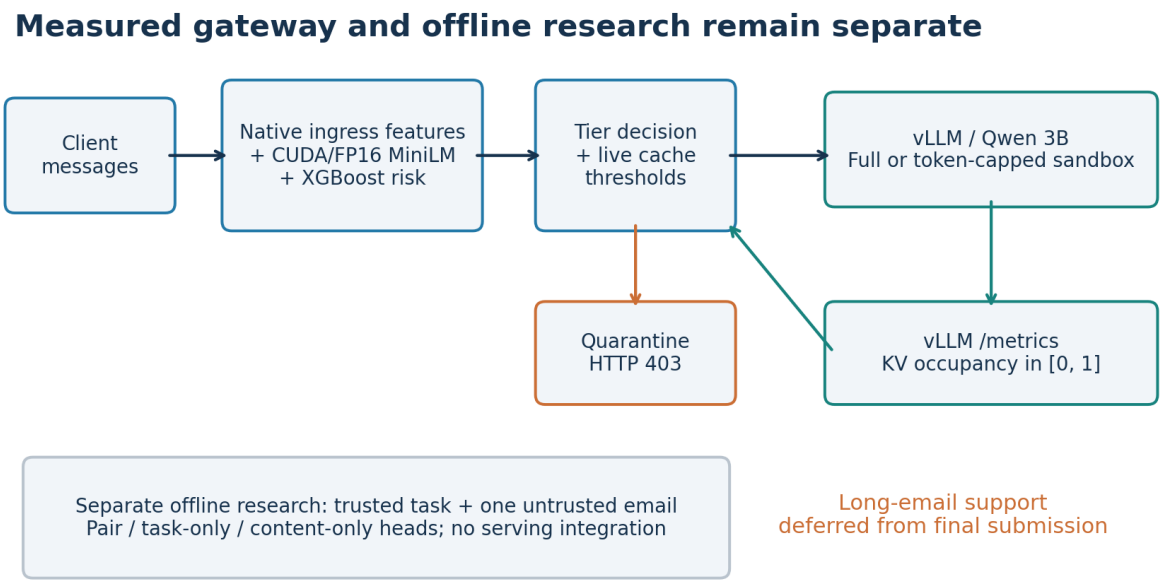


Figure 1: The live gateway and separate offline research pipeline. Experimental short-email heads have not replaced the serving classifier.

3 System design and implementation

3.1 Native ingress feature vector

The gateway forms the six-feature vector

$$x = [n_{tokens}, s_{lexical}, s_{semantic}, r_{repeat}, d_{output}, c_{session}].$$

Feature	Implemented interpretation	Limitation
Prompt length	`tiktoken` token count	Not the backend model's exact tokenizer budget
Lexical similarity	Maximum TF-IDF cosine similarity to reference phrases	Reference coverage is finite
Semantic similarity	Normalized MiniLM embedding similarity, with a 0.30 floor correction	A semantic match is not proof of malicious intent
Repetition ratio	Repeated word-bigram counts divided by bigram count	Does not cover every resource attack
Output demand	Token count plus regex-based demand multipliers	Heuristic, not a learned generation-length prediction
Session repeat count	Cumulative repeated-prefix/session counter	Not a measured requests-per-second window or decaying rate limiter

The multiclass native model predicts benign, injection and exhaustion probabilities. Risk is

$$R = 1 - P(benign).$$

The native lab classifier's SHA256 begins `25ea5790c5b0151e`. Its recorded 98.24% training accuracy is **resubstitution**, not held-out security accuracy. `features.py` also retains simulation-oriented utilities; the native extraction path used for live requests is in `app.py`.

3.2 Tier and pressure policy

Policy	Full	Sandbox	Quarantine
Normal/static	Risk below 0.35	0.35 to below 0.75	At least 0.75
High pressure or stale/unknown telemetry	Risk below 0.25	0.25 to below 0.65	At least 0.65

Adaptive mode tightens thresholds when cache occupancy is greater than 0.85. The implemented metric is `vllm:kv_cache_usage_perc`, parsed as a fraction in [0,1], including labeled series. A background thread polls approximately every second; samples older than five seconds are treated conservatively. Static mode preserves observed telemetry but uses the normal thresholds.

Quarantine returns HTTP 403 without forwarding that request for LLM generation. It still incurs gateway feature extraction, including CUDA embedding work. Sandbox admission caps output at 256 tokens and uses a five-second backend HTTP read timeout. This is not process/container isolation or a verified GPU execution deadline. Client-side timeout does not by itself prove cancellation of backend computation.

The gateway's current message selection and session accounting do not establish complete conversation/trust-boundary robustness. Those limits remain visible rather than being hidden by the project title.

4 Experimental environment and evidence provenance

Item	Recorded setup
Lab hardware	Ubuntu workstation; NVIDIA RTX A4000, 16 GB VRAM
Backend	Qwen/Qwen2.5-3B-Instruct via vLLM; separate backend environment
Gateway environment	Python 3.10.12; Sentence Transformers 3.4.1; Transformers 4.47.1
Native ML packages	scikit-learn 1.7.2; XGBoost 3.2.0; NumPy 2.2.6
Recorded PyTorch version	2.13.0 in later supplied runtime metadata; CUDA/FP16 device recorded
Backend launch configuration	FP16; prefix caching; 256 MiB KV pool; max model length 2,048; max running sequences four
Final local verification	123 regression tests passed in 30.224 seconds; four native artifact hashes matched

Version values above are preserved as reported. They were not independently re-snapshotted for each GPU replication. The controlled benchmark JSON records consistent cache configuration: 268,435,456 bytes, 7,280 tokens and 455 blocks. Complete startup logs were not supplied, limiting independent confirmation of every launch flag and process restart.

The lab operator executed GPU workloads and supplied JSON/log archives. Cloud analysis checked identities, hashes, counts, per-request usage, aggregate counters and recorded probabilities. It did not independently rerun physical GPU inference. Final software checks were run at source commit `e037df8`; the evidence consolidation baseline is `973d233`. Later report generation does not change the historical measurements.

5 GPU resource-protection experiment

5.1 Workload and controls

Three blocks compared direct vLLM, static gateway and adaptive gateway with balanced order: direct/static/adaptive, static/adaptive/direct, and adaptive/direct/static. There are nine runs, three per condition. The operator restarted services between conditions; initial idle/zero-counter observations were consistent with fresh state. Two excluded warmups preceded each measured run.

Each run offered eight concurrent demand requests, with a 1,792-token cap and `ignore_eos=true`, plus eight short benign probes staggered by three seconds. Benign requests capped output at 16 tokens. Temperature was zero. Direct-only runs still had a static gateway available for readiness/telemetry and the excluded warmup. These facts distinguish offered client concurrency from simultaneous GPU execution.

The controlled workload is capped synthetic generation demand. It is **not** a 100,000-word input experiment, natural unlimited generation or a demonstrated physical 16 GB exhaustion test.

5.2 Results

Condition	Benign run means, seconds (blocks 1/2/3)	Mean of run means	Benign completions	Demands blocked
Direct	47.709 / 55.629 / 47.393	50.244 s	24/24	0/24
Static	0.301 / 0.299 / 0.307	0.302 s	24/24	24/24
Adaptive	0.297 / 0.300 / 0.297	0.298 s	24/24	24/24

Direct runs reached 99.56–100% sampled KV occupancy and peak waiting queues of 11–12 requests. Both protected conditions peaked at 0.881% occupancy and recorded no waiting queue. All offered benign requests completed; direct service slowed substantially rather than universally failing or crashing. No unexpected measured outcome or telemetry error was recorded in the supplied replications.

The nine runs use the same benchmark source hash and native classifier hash. One client commit preceded the other eight; the intervening repository change did not change the benchmark/gateway code. Server source commit was not independently exposed. The first attempted block-3 direct invocation failed readiness before load; its subsequent successful measured run is retained. No measured failed trial was supplied and discarded.

5.3 Counter reconciliation and cost interpretation

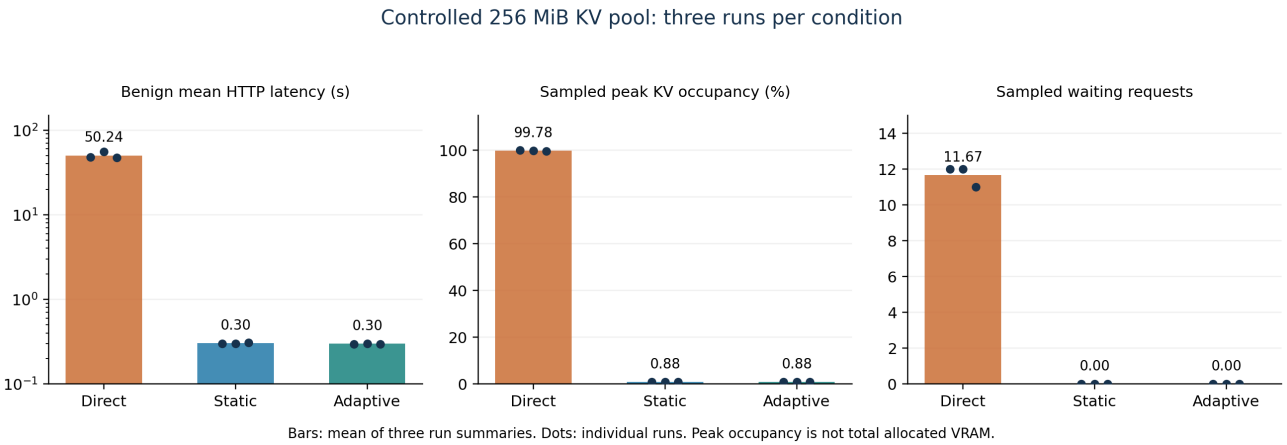


Figure 2: Native resource experiment. Each dot is one run. Cache occupancy describes the constrained KV pool, not total device allocation.

Per-run counter	Direct	Each protected condition	Avoided versus direct
Backend prompt tokens	912	296	616
Backend generated tokens	14,464	128	14,336
Backend successful generations	16	8	Eight demand generations

Aggregate prompt/generation/success deltas match successful HTTP usage in each run. Across three blocks, **each** protected condition avoided 1,848 prompt tokens and 43,008 generated tokens relative to the same three direct controls. The two protection comparisons must not be added as though six independent direct controls existed.

These counters measure avoided backend work for this workload. No energy meter, cloud bill or calibrated GPU-dollar measurement was collected. Quarantine does not imply zero total GPU FLOPs: the gateway itself embeds prompts. Sampled device allocation ranged approximately 7,403–7,509 MiB across conditions and was not reduced to zero. Allocation and KV occupancy are distinct measurements.

5.4 Adaptive-mechanism and latency boundaries

All measured adaptive decisions used the normal policy; all demands were already high risk. No protected adaptive run reached the tightening condition. Consequently these results show that both policies protect this workload, but **do not isolate a causal advantage of adaptive tightening**. Small differences between static and adaptive means, with only three repetitions, do not establish statistical superiority or equivalence.

There were only 24 benign probes per condition. No P99 or population tail-latency guarantee is estimated. Polling may miss brief peaks; thermal state, other processes, scheduling and prefix-cache effects were not fully controlled. Consistent sub-2 ms embeddings and sub-5 ms live ingress were not established. Extraction-only measurements must not be confused with classifier, HTTP or backend generation latency.

6 Security and contextual-model evaluation

6.1 Native and specialized diagnostics

The native model restricted only three of 75 constructed BIPIA-derived contexts: one quarantine and two sandbox decisions; 72 were full. This is low classification coverage on that diagnostic, not evidence that 72 attacks succeeded against the downstream LLM. A specialized shadow detector initially flagged all 75 attack contexts and none of 200 AI-labeled benign prompts, but also flagged the clean memo control. Matched-wrapper follow-up showed strong sensitivity to guard text and delimiters. Those failures prevent a perfect-detection claim, and the specialized detector was not integrated.

6.2 Task/content prototype and transfer

An offline pair representation used MiniLM task and content vectors:

$$z = [t, c, |t - c|, t \odot c, t^T c].$$

With 384-dimensional embeddings, the pair vector has 1,537 dimensions. Separate task-only and content-only heads provide ablations. These models are distinct from the serving six-feature native classifier.

A 400-case AI-authored/programmatic synthetic corpus split by topic into 280 training, 60 validation and 60 test cases. Templates were shared. Pair scored 59/60 on the topic test, task-only 48/60 and content-only 36/60. A subsequent 48-case wording-transfer diagnostic exposed poor generalization: pair reviewed four of 16 redirections, missed 12 and falsely reviewed one of 32 allowed cases. The retained negative result is more informative than presenting the earlier synthetic score as deployment accuracy.

7 External LLMail short-email study

7.1 Source, labels and preparation

LLMail-Inject provides adversarial submissions from a controlled email-assistant challenge. Human participants authored attacks, but annotation targets include API-trigger evidence and LLM-judge judgments. This is more varied than local templates; it is not production incident telemetry or independent human-adjudicated ground truth.

The pinned source revision was `1063bdf01ec8762b812d5e06ee768a06faa5a6f7`; the dataset card declares MIT. Phase 1 contained 370,724 raw submissions, 160,741 annotated unique texts, 224 teams and 203 publisher benign-email occurrences. Annotation normalization accepted literal True/False strings and JSON booleans without coercing numeric lookalikes. Totals were 127,376 True, 9,452 False, 23,911 Unclear and two malformed values. Unclear/malformed values were not forced into binary training targets.

Exact serialization joined 160,734 annotation texts to raw submissions; seven remained unmatched. Shared exact text connected teams into 142 components, largest 78 teams. Whole-component assignment produced 101,749/17,537/17,536 definite-label training/validation/test projections. The legitimate scenario `user_query`, not the attacker objective, was paired with one unchanged email. This omits the rest of the retrieval context and tool environment.

7.2 Long-input deferral and lexical selection

MiniLM's window is 256 tokens including special tokens, separately for task and full email. Before filtering, 59,924 training, 6,434 validation and 3,547 test emails exceeded the window. The

submission therefore evaluates only eligible short emails and explicitly **defers long-email handling**; there is no silent truncation or implied long-document coverage.

An initial lexical screen exhausted its comparison budget. The subsequent preparation completely removed cross-split normalized-text/token-bag exact collisions among short cases and applied approximate 64-permutation MinHash candidate screening with unigram Jaccard verification at 0.8. Oversized candidate sets were handled by conservative lower-priority exclusion rather than skipped. This may remove genuine diversity, and approximate screening cannot prove semantic/template independence.

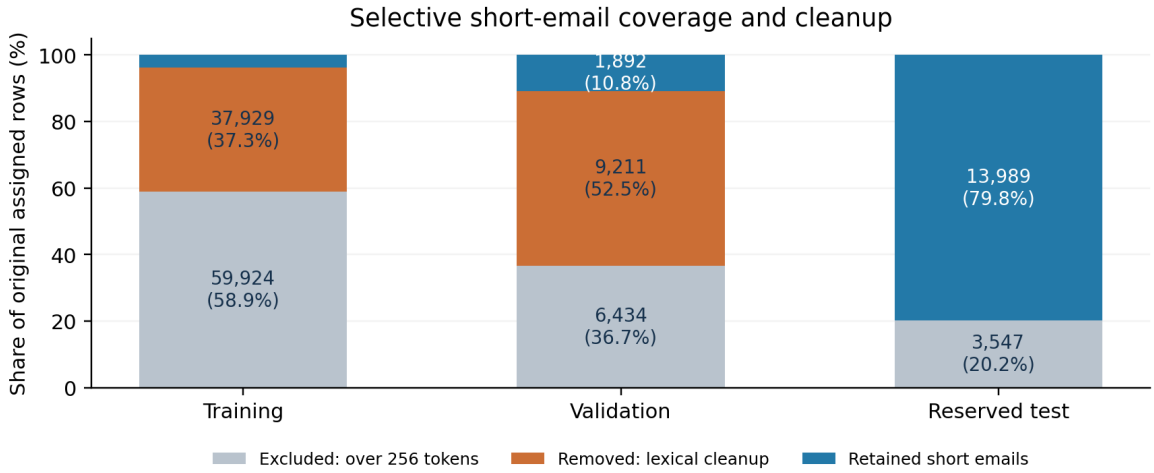


Figure 3: Selection materially changes coverage and development distribution. Lexical removals are not all confirmed duplicates.

Retained split	Source True	Source False	Total
Training	3,648	248	3,896
Validation	1,852	40	1,892
Reserved Phase 1 test	13,282	707	13,989

Cleanup removed 47,140 of 66,917 eligible short rows (70.45%). These are not 47,140 verified duplicates. Test-priority preservation and heavy development removal introduce selection bias. The reserved rows span 69 team components and are not 13,989 independent authoring units.

7.3 Fixed training, cutoff selection and reserved results

Three XGBoost heads used fixed 150-tree/depth-four settings, seed 42 and inverse-frequency training weights. MiniLM ran CUDA/FP16 without encoder fine-tuning. The original cutoff was 0.5. Training used 3,896 rows and reported validation separately; zero reserved inputs were encoded during fitting.

Pair validation balanced accuracy was 55.42%, with 34/40 source negatives flagged. Content-only was 56.78%, with 33/40 flagged. An always-positive rule already achieved 97.89% raw validation accuracy and 50% balanced accuracy, demonstrating why raw accuracy alone is misleading.

A later frozen operating-cutoff rule selected the smallest cutoff at least 0.5 that flagged at most four of 40 validation source negatives. Numerical cutoffs used validation negatives only; the rule was designed after inspecting development diagnostics. This is operating-point selection, not probability calibration. Cutoffs were frozen before reserved scoring.

Head	Cutoff	Positive recall	Negative specificity	Balanced accuracy	False flags / 707 source negatives
Pair	0.5	98.06%	21.22%	59.64%	557
Pair	0.95396733	61.59%	76.66%	69.13%	165
Task only	0.5	93.25%	6.51%	49.88%	661
Task only	0.75159937	0.14%	99.43%	49.79%	4
Content only	0.5	97.42%	30.41%	63.91%	492
Content only	0.95745611	54.46%	80.06%	67.26%	141

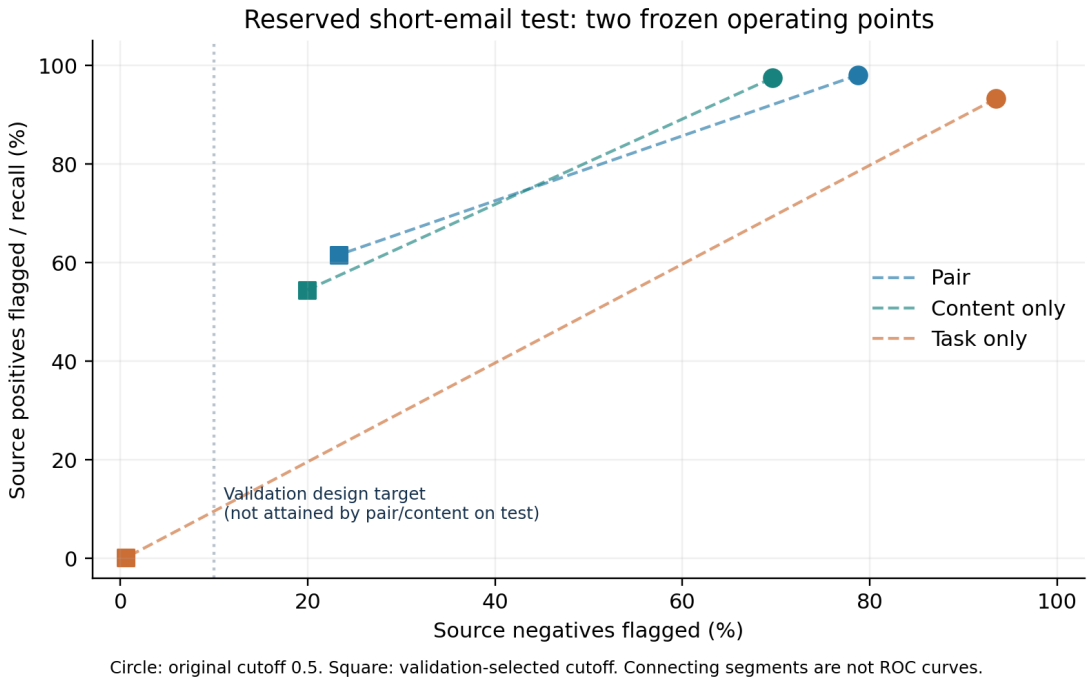


Figure 4: The original and adjusted operating points on reserved source labels. The validation 10% negative-flag target did not transfer for the pair or content-only head.

At the adjusted cutoff, pair missed 5,101 source positives and content-only missed 6,049. Pair's adjusted balanced accuracy was about 1.87 percentage points higher, while content-only had the higher ROC AUC (0.7790 versus 0.7631). These observations do not demonstrate consistent broad superiority of pairing. The empirical 90% validation specificity goal fell to 76.66% pair and 80.06% content-only on the reserved data.

Source-negative flags are not automatically confirmed real-world false positives, and source-positive misses are not measured successful downstream attacks. The test was reserved from fitting/numerical cutoff selection but participated in documented dataset QA and priority cleanup. It is now inspected and cannot become fresh evidence after further tuning.

8 Benign checks, labeling policy and reviewer provenance

The separate publisher benign collection contained 203 occurrences, reduced to 160 unique eligible emails after 43 duplicate exclusions. At cutoff 0.5, pair flagged 20/160 (12.5%), content-only

eight (5%) and task-only all 160. Adjusted cutoffs flagged none of those already inspected emails; that recheck is reused diagnostic evidence, not independent confirmation.

A blinded 80-case validation packet selected all 40 source negatives and 40 hash-selected source positives. AI-only review labeled 24 benign-for-injection, 42 injection and 14 unclear. Among the 40 source negatives, AI labeled 11 injection, 24 benign and five unclear. This establishes disagreement with an AI task-aware interpretation, not proof that the publisher is wrong.

The final 25-case priority review records reviewer Reet, with 13 injection, six benign and six unclear judgments. Twenty-four preserve original task/email exactly; one garbled-source entry was replaced by a placeholder and is excluded. Accepted intact cases therefore contain 13 injection, six benign and five unclear; only 19 clear cases support diagnostic score comparison. Prior AI exposure and targeted selection prevent an independent blinded-ground-truth claim. These labels were not used to overwrite the publisher data or refit a head.

The rubric separates recipient-facing email requests from assistant-directed task replacement, role impersonation and unauthorized tool demands. Phishing, harmful topics, encoding and corruption do not automatically determine the injection label. Ambiguous cases remain unclear. A small reviewed development packet cannot repair the entire source dataset or support population accuracy claims.

9 Findings, limitations and deferred work

Supported: the working gateway prevented the tested admitted demand workload from reaching the backend and preserved benign mean latency under a constrained KV pool. Recorded counters substantiate workload-specific avoided backend tokens. Software regressions and artifact integrity passed the final local checks.

Not established: universal injection prevention, isolated adaptive-threshold benefit, full-default-cache OOM prevention, GPU cancellation on timeout, P99 latency, calibrated monetary/energy savings, consistent ingress latency targets, complete conversation/session robustness, or general long-email protection.

Long-email processing is deliberately deferred to protect the five-hour submission window. Implementing chunk aggregation, training with weak per-email labels, preserving attacks across chunk boundaries and evaluating a full-length corpus would require separate design and evidence. The short-only scope must stay explicit in all summaries.

Other validity limits include only three GPU repetitions per condition, nonrandom balanced run order, operator-supplied rather than independently executed lab evidence, limited startup records, sampled telemetry, automated public labels, heavy conservative filtering, small negative validation count, related attack families, prior review exposure and unknown pretraining contamination. Public availability does not make data independent or labels correct.

No experimental injection head has replaced the native serving classifier. Existing endpoint forwarding, output caps and telemetry behavior are retained. The report is an engineering/research evaluation, not a production safety certificate or a patent-novelty determination.

10 Conclusion

The project reaches a defensible prototype milestone: GPU-backed routing, live cache telemetry, repeatable controlled admission protection and an auditable evaluation trail. It also demonstrates that strong training/synthetic scores and cutoff adjustments do not by themselves yield robust injection protection on varied external inputs.

The immediate deliverable is this evidence-based report and reproducibility package. If additional bounded Phase 2 testing completes within the remaining window, it should appear as a clearly versioned addendum using frozen models/cutoffs and overlap reporting. It must not be

assumed successful in advance. Further research should prioritize consistent task/trust targets, independently reviewed evaluation and full-context coverage before deployment claims.

11 Reproducibility and evidence index

The source repository is reetshrivastav/cad_project. Paths below refer to preserved repository evidence. A machine-readable index with SHA256 digests is supplied in the report bundle.

- **E1:** GPU replications, per-run CSV and reconciled counters.
- **E2:** Sandbox validation, output cap and backend counters.
- **E3:** Context-model wording transfer, including negative results.
- **E4:** LLMail short-model training, fixed settings and artifacts.
- **E5:** Reserved LLMail evaluation, both frozen cutoff policies.
- **E6:** Benign and cutoff diagnostic, reused-evidence boundaries.
- **E7:** Label alignment diagnostic; corrected reviewer records are indexed in the bundle.
- **E8:** Final lab check summary, software/artifact verification.

The report figures are generated offline from retained CSV/JSON files; figure source digests accompany the assets. No new benchmark is implied by rendering figures or producing this document.

To check software/artifacts locally with servers stopped:

```
.venv/bin/python scripts/final_project_check.py \
  --model-dir models/lab_native_v1 \
  --output-dir results/project_handoff_new
```

The final retained run passed 123 tests in 30.224 seconds and matched all four native artifact hashes without deserializing models. Illustrative simulation cost constants are not used as measured billing data in this report.

For a live demonstration, use the backend launcher `start_lab_vllm.sh` and the gateway launcher `start_lab_gateway.sh adaptive`, both in the scripts directory. Terminal 3 can query `/api/status` and `/api/pressure`. Follow the pinned environment setup and controlled-cache scope. Experimental offline heads must not be substituted silently.

12 Selected technical references

1. Kwon et al. *Efficient Memory Management for Large Language Model Serving with PagedAttention*. arXiv:2309.06180. Background for vLLM cache management, not proof of this gateway's results.
2. Chen and Guestrin. *XGBoost: A Scalable Tree Boosting System*. arXiv:1603.02754.
3. Reimers and Gurevych. *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. arXiv:1908.10084.
4. Abdelnabi et al. *LLMail-Inject: A Dataset from a Realistic Adaptive Prompt Injection Challenge*. arXiv:2506.09956; publisher dataset.
5. *Benchmarking and Defending Against Indirect Prompt Injection Attacks on Large Language Models* (BIPIA). arXiv:2312.14197; source repository.
6. Debenedetti et al. *AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents*. arXiv:2406.13352. Relevant future agent-level evaluation, not executed here.

Preparation disclosure: lab runs and reviewer judgments were supplied by the project operator. AI assistance supported code development, offline evidence reconciliation, diagnostic labeling with explicit provenance, figure generation and report preparation. Raw labels and negative results were preserved; human and AI judgments are not merged into a claimed independent ground-truth test set.